Analisi delle intrusioni

Description

Scoperta di un'intrusione

- Un attaccante segue una sequenza di azioni per ottenere determinati privilegi in un sistema.
- Per migliorare la sicurezza, dobbiamo conoscere tutte le possibili intrusioni.
- Se modifichiamo il sistema solo per bloccare intrusioni già note, il rischio di successo per nuove intrusioni può aumentare (applicazione del Teorema di Bayes).
- Questo spiega perché valutare la sicurezza basandosi solo su un insieme limitato di intrusioni (come fa un red team) può essere insufficiente.

Teorema di Bayes applicato alla sicurezza

- È spiegato con l’analogia dei ponti su un fiume: aumentare il numero di ponti aumenta le possibilità di attraversamento, ma può anche aumentare il traffico e quindi il tempo medio di percorrenza.
- In sicurezza, più percorsi di attacco conosciuti non significano sempre maggiore protezione: se ci si concentra solo su alcuni, altri meno noti possono diventare più efficaci.

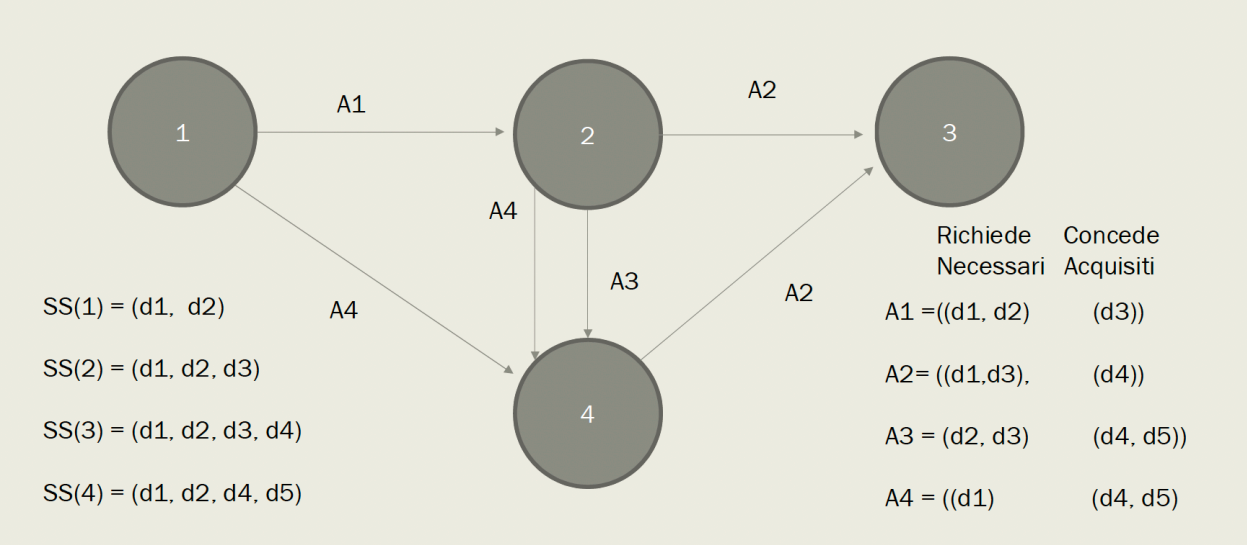
Costruzione = scoperta di un’intrusione

- Per scoprire le intrusioni si usa l'**adversary emulation**, che riproduce il comportamento di un attaccante.
- Un attaccante parte da **diritti iniziali** (legali) e cerca di ottenere **diritti finali** (privilegi di alto livello).
- Deve alternare pianificazione e attacchi, poiché scopre nuove informazioni strada facendo.

- Gli insider sono i più pericolosi perché possiedono già le informazioni necessarie.

Attack Graph

- È un grafo orientato in cui:
  - I **nodi** rappresentano insiemi di diritti posseduti dall'attaccante.
  - Gli **archi** sono attacchi eseguibili con i diritti attuali.
- L'attaccante è razionale e non ripete attacchi inutili.
- Un nodo iniziale rappresenta i diritti di partenza, mentre un nodo finale è uno stato in cui l'attaccante ha raggiunto il suo obiettivo.
- Diversi cammini nel grafo corrispondono a diverse intrusioni.
- Per ottenere un **intrusion graph**, si aggiungono anche le azioni necessarie per scoprire le informazioni utili agli attacchi.



Proprietà dell' **Attack graph:**

- Acquisizione dei diritti è monotona perchè AG modella solo i possibili attacchi e

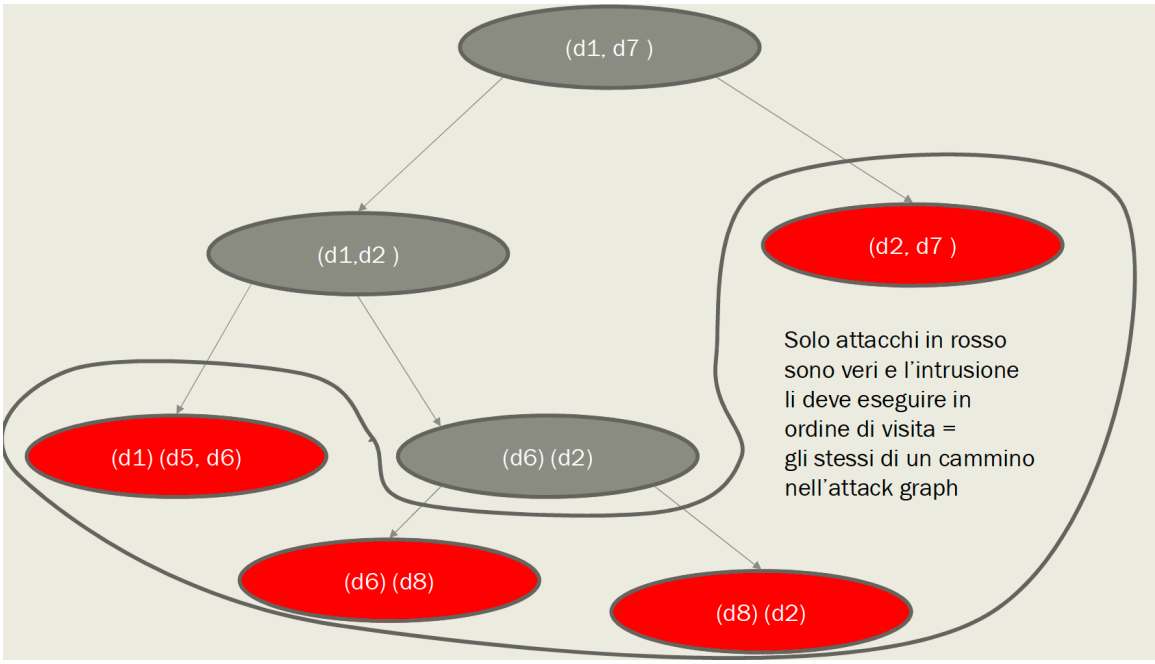
non le possibili azioni di difesa che potrebbero riportare indietro attaccante

- Monotonia implica che in un nodo si possono eseguire tutti gli attacchi possibili
  nel cammino fino a quel nodo purchè
  - Non siano già stati eseguiti
  - Concedano almeno un diritto che attaccante ancora non possiede
- Complessivamente acquisizione monotona porta a grafi molto ricchi e di difficile
  costruzione nel caso di sistemi ragionevolmente complessi
- Più dettagliata la modellazione dei diritti, maggiore è la complessità del grafo e della sua costruzione
- Conviene considerare non tutti i possibili cammini ma solo quelli che la strategia
  dell'attaccante potrà scegliere
- Difensore può costruirlo subito, attaccante alterna costruzione ed attacchi
- Possiamo costruire a posteriori un attack graph che descrive gli attacchi in una intrusione
- In generale in questo grafo avremo anche attacchi inutili perché l'attaccante non aveva le
  informazioni che gli permettevano di capire che l'attacco era inutile
- Quindi in generale attack graph a posteriori
  - Non contiene cicli
  - Non contiene attacchi che concedono diritti che l'attaccante già possiede
  - Può contenere attacchi inutili ovvero che concedono diritti che non servono per
  - raggiungere un obiettivo
- Nasconde informazioni
  - sulla probabilità di successo di un attacco
  - sul tempo necessario per eseguire un attacco

- Queste informazioni permettono di calcolare il tempo medio per passare da un nodo
  ad un altro
- Quindi sono utili
  - per ottimizzare costi e tempi per attaccante
  - per permettere al difensore di scoprire percorsi più brevi e quindi più pericolosi

Attack Tree (Intrusion Tree)

- È una versione semplificata dell'**Attack Graph**.
- Rappresenta un'intrusione come un **albero**, dove:
  - I **nodi** rappresentano attacchi.
  - I **sottoalberi** mostrano come un attacco può essere eseguito.
- Se un attacco è già noto, non viene ulteriormente scomposto.
- Le **foglie** rappresentano la frontiera, ossia i punti finali dell'attacco.
- Può essere una versione **AND/OR**:
  - **AND**: tutti gli attacchi nei nodi figli devono essere eseguiti.
  - **OR**: basta un attacco per proseguire.
- È utile per rappresentare la struttura degli attacchi, ma non aiuta nella loro progettazione.



Automazione delle Intrusioni

- Le intrusioni possono essere **automatizzate**.
- In passato erano **manuali**, ora vengono automatizzate in vari step.
- Caso tipico: **ransomware**, con automazione di:
  1. **Accesso iniziale** al sistema.
  2. **Esplorazione e raccolta informazioni**.
  3. **Esecuzione del ransomware**.
- Esiste una **specializzazione del lavoro tra gruppi criminali**, con team diversi che si occupano di fasi specifiche dell'attacco.

Fermare un'Intrusione

- Dato un **Attack Graph**, possiamo fermare un attacco tagliando il grafo nei punti giusti.
- Fermare un attacco interrompe la catena dell'intrusione.
- Se blocchiamo un attacco che compare in più intrusioni, fermiamo **tutte** quelle intrusioni.

- Il problema diventa **ottimizzare** la difesa:
  - **Troviamo il minor numero di attacchi da fermare** per bloccare tutte le intrusioni.
  - Oppure scegliamo attacchi da fermare con il **costo minimo**.

Ranking Contestuale delle Vulnerabilità

- Il metodo precedente aiuta a **dare un punteggio alle vulnerabilità** in base al numero di intrusioni che si possono fermare eliminandole.
- Questo metodo è **specifico per un sistema**, a differenza del CVSS che valuta solo le caratteristiche della vulnerabilità.
- Possiamo anche definire un **ordine ottimale** per applicare le contromisure:
  - Iniziamo eliminando la vulnerabilità che compare nel **maggior numero di intrusioni** ancora attive.

Persistenza dell'Attaccante

- Gli attaccanti cercano di **rimanere nel sistema** per continuare a rubare dati o eseguire attacchi futuri.
- Possono creare **account nascosti**, installare **backdoor** o usare un'infrastruttura di **Command & Control (C2)**.
- La **C2 infrastructure** è simile a una **botnet** e viene usata per controllare i dispositivi compromessi.
- Gli attacchi che servono a creare questa infrastruttura sono **meno rumorosi**, quindi più difficili da rilevare.
- Per evitare il tracciamento, i moduli sul target comunicano con **nodi diversi della C2 infrastructure** (tecnica **fast flux**).
- Il **monitoraggio delle query DNS** può aiutare a individuare queste tecniche.

Strategia per Fermare le Intrusioni

1. **Costruire l'Attack Graph** per capire come si sviluppano le intrusioni.

2. **Rimuovere cammini inutili** (ottimizzazione).
3. **Applicare un "taglio" nel grafo**, ossia:
    - Eliminare le vulnerabilità critiche nel sistema reale (patch, controlli di sicurezza).
    - In questo modo, i nodi finali diventano **irraggiungibili**, impedendo il completamento dell'intrusione.

Per fermare un'intrusione utilizzando un **Attack Graph**, i tre passi fondamentali sono:

1. Costruzione dell'Attack Graph

- Creiamo un **grafo orientato** che rappresenta il sistema e l'attaccante.
- Ogni nodo rappresenta un **security status** (insieme di diritti dell'attaccante in quel punto).
- Ogni arco rappresenta un **attacco** che porta da uno stato di sicurezza all'altro.
- Il nodo iniziale è lo stato in cui l'attaccante ha solo i suoi **diritti legali**.
- I nodi finali rappresentano gli stati in cui l'attaccante ha **raggiunto i suoi obiettivi**.

2. Eliminazione dei cammini inutili

- Non tutti i percorsi del grafo sono **realisticamente percorribili**.
- Si rimuovono i **nodi e archi ridondanti**, concentrandosi solo sulle intrusioni effettivamente possibili.
- Questo permette di **ridurre la complessità** e focalizzarsi sulle reali minacce.

3. Scelta ed applicazione del taglio

- Dobbiamo trovare un **insieme minimo di attacchi da bloccare** per fermare tutte le intrusioni.
- Scegliamo le vulnerabilità da eliminare in modo da **rimuovere gli archi critici** nel grafo.

- Nell’ambiente reale, questo significa applicare **patch, controlli di sicurezza o altre contromisure**.
- Il risultato è che i nodi finali (obiettivi dell’attaccante) diventano **irraggiungibili**.